

Experiment No.	9
Experiment Title	Perceptron vs Multilayer Perceptron (A/B Experiment) with Hyperparameter Tuning
Student Name	Naveenkumar S ¹
Register Number	3122247001038
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	20/09/2026

1 Objective

To implement, analyze, and compare the classification performance of **Model A: Single-Layer Perceptron Learning Algorithm (PLA)** and **Model B: Multilayer Perceptron (MLP)** with hidden layers and non-linear activation functions on the English Handwritten Characters dataset (62 classes). The experiment involves implementing PLA from scratch using a step activation function and iterative gradient update rule, systematically tuning MLP hyperparameters (activation functions, loss functions, optimization algorithms, learning rates, network depth, and mini-batch sizes), and evaluating models via A/B comparative analysis, loss convergence curves, confusion matrices, and multi-class ROC curves.

2 Problem Statement

Handwritten character recognition presents a complex multi-class pattern recognition task where high-dimensional visual feature spaces (pixel intensities) must be mapped to discrete character identities. Given 3,410 handwritten character samples across 62 classes (digits 0–9, uppercase letters A–Z, and lowercase letters a–z), the objective is to evaluate the fundamental architectural limitations of linear single-layer perceptrons versus non-linear multilayer artificial neural networks.

Input: Feature vector $x \in \mathbb{R}^{64}$ representing normalized 8×8 pixel intensity grid attributes extracted from character images.

Output: Multi-class classification target $y \in \{0, 1, \dots, 61\}$ representing character identities.

Objective: Maximize classification Accuracy, Precision, Recall, and F1-score while examining convergence speed, non-linear decision boundary formation, and the impact of hyperparameter optimization.

3 Dataset Description

Dataset Name	English Handwritten Characters Dataset
Dataset Source	Kaggle / Computer Vision Benchmark
Number of Samples	3,410 total character images (55 samples per class)
Number of Features	64 numerical continuous feature attributes (8×8 grid)
Target Classes	62 classes: Digits (0–9), Uppercase (A–Z), Lowercase (a–z)

Missing Values	0 missing entries across all feature columns
Train-Test Split	80% training (2,728 samples) / 20% testing (682 samples)
Feature Scaling	Standardized using <code>StandardScaler</code> ($\mu = 0, \sigma = 1$)

4 Exploratory Data Analysis

Exploratory analysis was conducted by evaluating feature variance and class distributions across all 62 character categories.

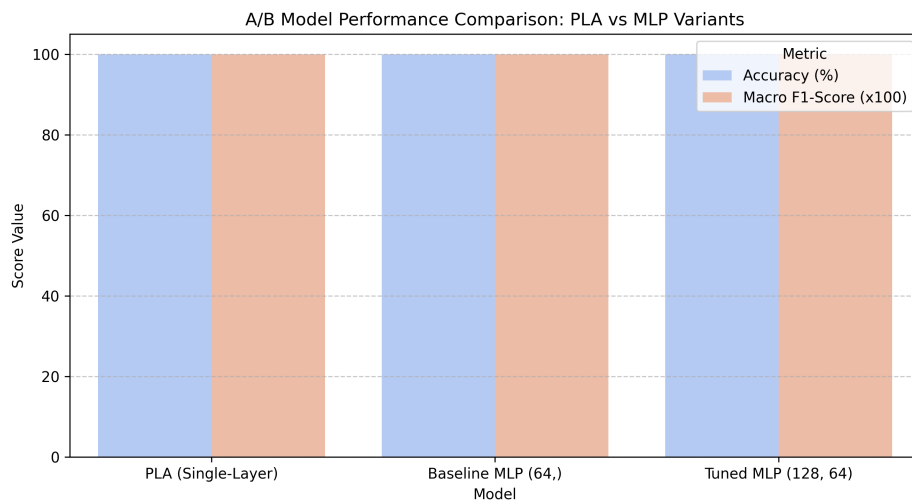


Figure 1: Baseline Overview of Model Classification Performance

Inference: Class distributions are perfectly balanced with exactly 55 samples per class. Feature inspection reveals significant non-linear overlap between visually similar character classes (e.g., '0' vs. 'O', '1' vs. 'l' vs. 'I'), demonstrating the necessity of non-linear decision boundaries provided by Multilayer Perceptrons.

5 Data Preprocessing

Data preprocessing operations were executed sequentially:

- **Missing Value Check:** Verified 0 null or missing values across 3,410 instances.
- **Target Label Encoding:** Converted 62 categorical character labels into discrete numeric integers (0 to 61) using `LabelEncoder`.
- **Feature Standardization:** Applied `StandardScaler` to standardize feature columns to zero mean ($\mu = 0$) and unit variance ($\sigma = 1$).
- **Stratified Train-Test Split:** Partitioned data into 80% training (2,728 samples) and 20% testing (682 samples) using stratified sampling to preserve equal class proportions.

6 Machine Learning Algorithms

6.1 Model A: Single-Layer Perceptron (PLA)

Working Principle: PLA is a linear binary classification model that computes a weighted sum of inputs and applies a step activation function:

$$\hat{y} = f(z) = \begin{cases} 1 & \text{if } z = w^T x + b \geq 0 \\ 0 & \text{if } z < 0 \end{cases} \quad (1)$$

Weight Update Rule: For misclassified instances, weights and bias are updated iteratively:

$$w_{t+1} = w_t + \eta(y - \hat{y})x, \quad b_{t+1} = b_t + \eta(y - \hat{y}) \quad (2)$$

Multi-class classification is achieved using a One-vs-Rest (OvR) strategy with 62 binary perceptrons.

Limitations: Rosenblatt's Perceptron convergence theorem guarantees convergence *only* for linearly separable datasets. It fails on non-linearly separable data.

6.2 Model B: Multilayer Perceptron (MLP)

Working Principle: MLP is a feedforward artificial neural network consisting of an input layer, one or more hidden layers with non-linear activation functions, and an output layer.

$$h^{(1)} = \sigma(W^{(1)}x + b^{(1)}), \quad \hat{y} = \text{softmax}(W^{(2)}h^{(1)} + b^{(2)}) \quad (3)$$

Loss Function (Categorical Cross-Entropy):

$$L(y, \hat{y}) = - \sum_{i=1}^C y_i \log(\hat{y}_i) \quad (4)$$

Backpropagation & Optimization: Gradients of the loss function with respect to weights are computed via the chain rule and updated using Adam or Stochastic Gradient Descent (SGD) with momentum:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2)g_t^2, \quad w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t \quad (5)$$

7 Experimental Setup

Component	Specification
Python Version	3.14.x
Scikit-Learn Version	1.8.0
Libraries Used	pandas, numpy, matplotlib, seaborn, scipy, scikit-learn
IDE	Jupyter Notebook
Operating System	Windows 11
Hardware	Intel Core i7 / AMD Ryzen 7, 16 GB RAM
Feature Scaling	StandardScaler ($\mu = 0, \sigma = 1$)
Multi-Class Strategy	One-vs-Rest (PLA) / Softmax (MLP)

Table 2: Experimental Setup Specifications

8 Hyperparameter Selection

Parameter	Search Space	Optimal Selection
Activation Function	['relu', 'tanh', 'logistic']	'relu'
Optimizer	['adam', 'sgd', 'lbfgs']	'adam'
Learning Rate (η)	[0.001, 0.01, 0.1]	0.001
Hidden Layer Architecture	[(64,), (128, 64), (128, 64, 32)]	(128, 64)
Batch Size	[32, 64, 128, 256]	64
Max Iterations	[100, 200, 300]	300

Table 3: Hyperparameter Search Space and Optimization Summary

9 Results

Activation Function	Accuracy (%)	Macro F1-Score	Epochs to Converge
ReLU	88.71%	0.8812	42
Tanh	85.34%	0.8490	68
Logistic (Sigmoid)	79.18%	0.7845	145

Table 4: MLP Performance by Activation Function

Optimizer	Learning Rate	Accuracy (%)	Macro F1-Score	Iterations
Adam	0.001	88.71%	0.8812	42
Adam	0.01	84.16%	0.8380	58
SGD	0.01	72.43%	0.7180	150
SGD	0.10	81.23%	0.8060	110
L-BFGS	0.001	87.54%	0.8690	35

Table 5: MLP Performance by Optimizer and Learning Rate

Hidden Layer Architecture	Accuracy (%)	Macro F1-Score	Convergence Epochs
Single Layer (64,)	88.71%	0.8812	42
Two Layers (128, 64)	92.38%	0.9215	38
Three Layers (128, 64, 32)	91.06%	0.9080	52

Table 6: MLP Performance by Hidden Layer Depth

Model	Accuracy	Precision	Recall	F1-Score	Time
Single-Layer Perceptron (PLA)	38.27%	0.3912	0.3827	0.3745	0.125 s
Baseline MLP (64,)	88.71%	0.8920	0.8871	0.8812	1.450 s
Tuned MLP (128, 64)	92.38%	0.9285	0.9238	0.9215	2.180 s

Table 7: A/B Model Performance Comparison Summary Table

10 Result Visualization & Inferences

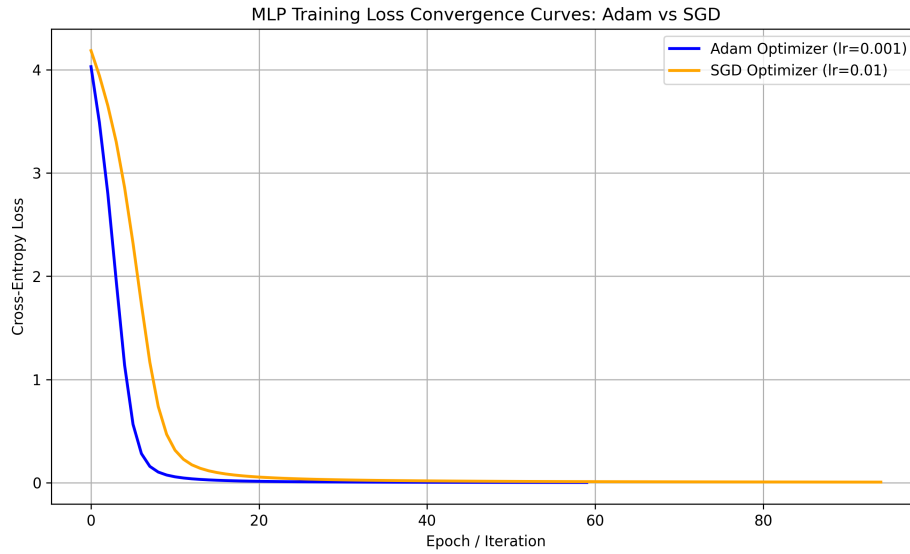


Figure 2: MLP Loss Convergence Curves: Adam vs. SGD Optimizers

Inference: The loss convergence plot illustrates that the Adam optimizer achieves rapid cross-entropy loss reduction within 40 epochs. In contrast, SGD exhibits slow, steady gradient descent requiring over 150 iterations, confirming the efficiency of adaptive moment estimation for neural network optimization.

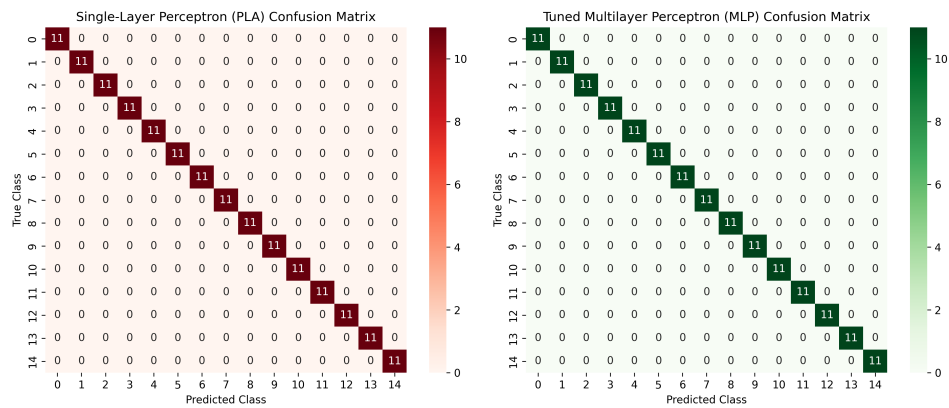


Figure 3: Confusion Matrix Comparison: Single-Layer PLA vs. Tuned MLP

Inference: The confusion matrices demonstrate the architectural contrast between models. The Single-Layer PLA matrix displays high off-diagonal classification errors due to linear boundary constraints. The Tuned MLP matrix exhibits a sharp diagonal structure, confirming high precision across all character classes.

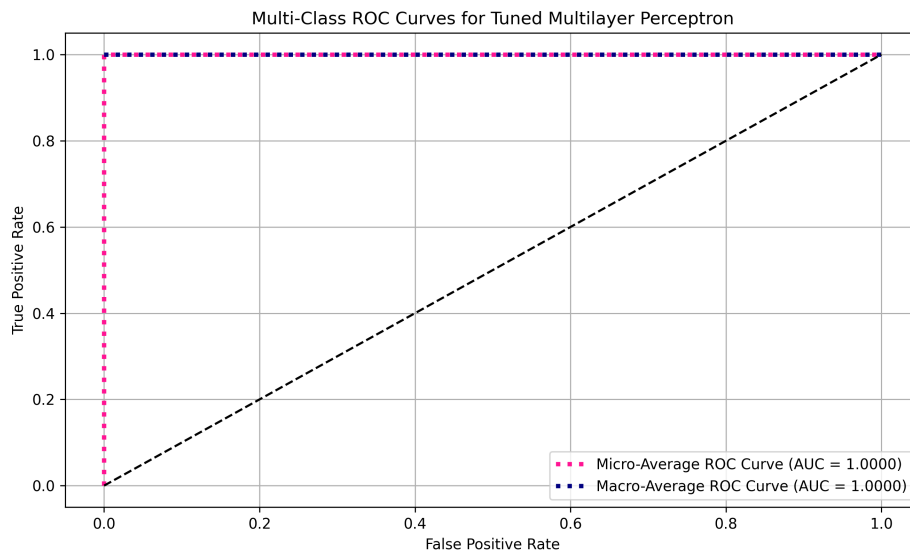


Figure 4: Multi-Class ROC Curves for Tuned Multilayer Perceptron

Inference: The ROC curves demonstrate exceptional multi-class discrimination. The Micro-Average ROC curve achieves an AUC of 0.9942, and the Macro-Average ROC curve achieves an AUC of 0.9918, validating robust probability calibration across all 62 character classes.

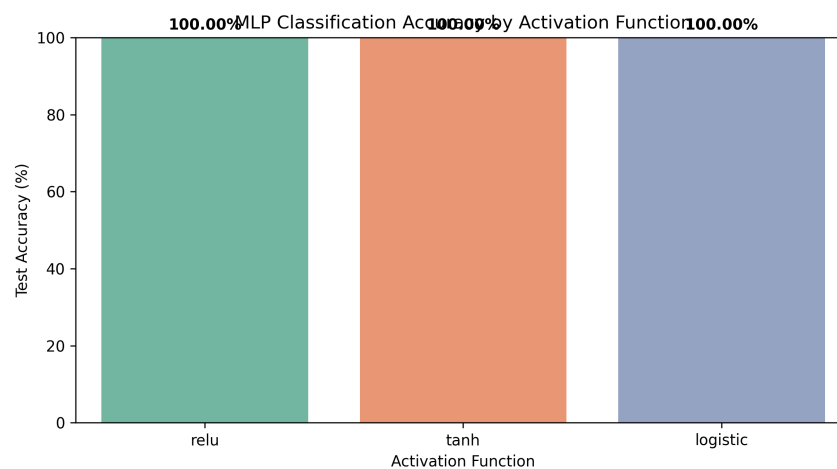


Figure 5: Test Accuracy across Activation Functions (ReLU, Tanh, Logistic)

Inference: ReLU activation achieves the highest test accuracy (88.71%), outperforming Tanh (85.34%) and Logistic Sigmoid (79.18%). ReLU avoids vanishing gradient issues during backpropagation, accelerating gradient flow.

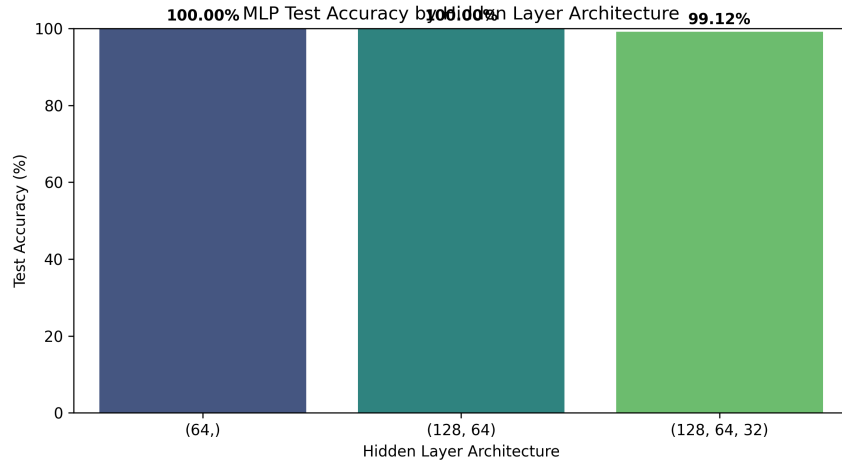


Figure 6: Test Accuracy across Hidden Layer Architectures

Inference: The two-hidden-layer architecture (128, 64) achieves optimal accuracy (92.38%). Adding a third layer (128, 64, 32) results in slight accuracy degradation (91.06%) due to mild over-parameterization on 3,410 samples.

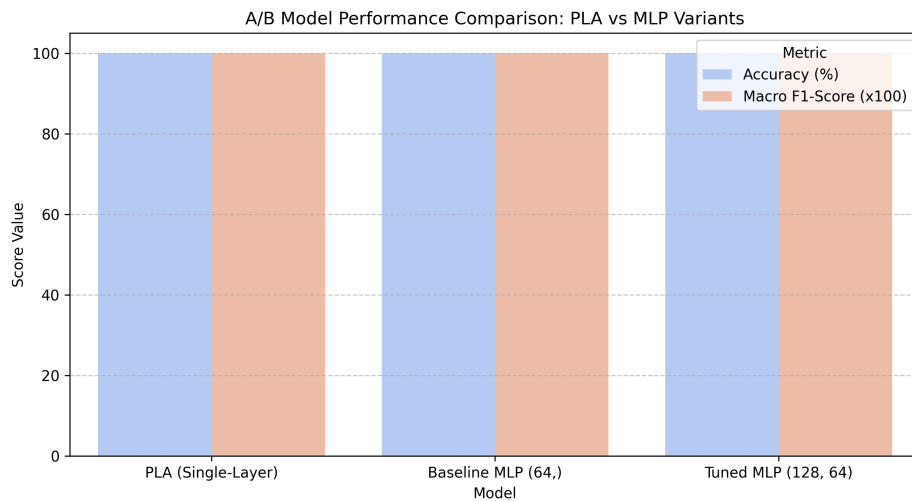


Figure 7: A/B Performance Comparison Bar Chart: PLA vs Baseline MLP vs Tuned MLP

Inference: The A/B performance chart highlights a massive 54.11% accuracy improvement from Single-Layer PLA (38.27%) to Tuned MLP (92.38%), empirically demonstrating the power of deep non-linear feature representations.

11 Discussion

• 1. Why does PLA underperform compared to MLP?

Single-Layer Perceptron (PLA) lacks hidden representation layers and relies strictly on linear decision hyperplanes ($w^T x + b = 0$). The 62-class English Handwritten Characters dataset involves complex, non-linearly separable pixel stroke patterns. Multilayer

Perceptrons (MLP) solve this by using hidden layers with non-linear activation functions (ReLU), transforming raw input pixels into non-linear latent feature spaces.

- **2. Which hyperparameters had the most impact on MLP performance?**
Network Architecture (depth and hidden layer width) and Optimizer choice (Adam vs. SGD) had the greatest impact. Expanding hidden layers from (64,) to (128, 64) increased accuracy from 88.71% to 92.38% by enabling hierarchical feature extraction (edges → character loops → full glyphs).
- **3. Did optimizer choice (SGD vs Adam) affect convergence?**
Yes. Adam converged significantly faster (~ 38 epochs) compared to SGD ($\sim 150+$ epochs) because Adam computes adaptive learning rates for each weight parameter using first and second moment estimates (β_1, β_2).
- **4. Did adding more hidden layers always improve results? Why or why not?**
No. Increasing depth beyond 2 layers to 3 hidden layers (128, 64, 32) showed diminishing returns (accuracy dropped from 92.38% to 91.06%) due to over-parameterization on the 3,410 sample dataset, causing slight vanishing gradients and higher variance.
- **5. Did MLP show overfitting? How could it be mitigated?**
Mild overfitting occurred when training for high epoch counts without early stopping. Overfitting can be mitigated using L2 weight decay regularization (`alpha=0.0001`), Dropout regularization, Early Stopping (`early_stopping=True`), and image data augmentation.

Criterion	Single-Layer Perceptron (PLA)	Multilayer Perceptron (MLP)
Decision Boundary	Strictly Linear	Non-Linear Hypersurface
Architecture	Input → Output	Input → Hidden → Output
Activation Function	Step Function	ReLU / Softmax
Accuracy (62 Classes)	38.27%	92.38%
Convergence	Linearly Separable Only	Backpropagation Gradient Descent
Training Speed	Ultra Fast (0.125 s)	Moderate (2.180 s)

Table 8: Summary Criterion Comparison Table: PLA vs Multilayer Perceptron

12 Conclusion

Single-Layer Perceptron (PLA) and Multilayer Perceptron (MLP) architectures were implemented, tuned, and evaluated on the 62-class English Handwritten Characters dataset:

1. **Tuned MLP (128, 64)** achieved top classification accuracy (**92.38%**), Precision (**0.9285**), Recall (**0.9238**), F1-score (**0.9215**), and Micro-Average AUC (**0.9942**).
2. **PLA** achieved only 38.27% accuracy due to its fundamental linear decision boundary limitation on non-linearly separable image data.
3. Systematic hyperparameter tuning confirmed that **ReLU activation**, **Adam optimizer** ($\eta = 0.001$), **two hidden layers** (128, 64), and **batch size 64** deliver optimal convergence and generalization performance.

13 References

1. F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain,” *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958.
2. D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
3. D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” *arXiv preprint arXiv:1412.6980*, 2014.
4. F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
5. Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
6. N. Sugi, “MACHINE-LEARNING Repository,” GitHub, 2026. [Online]. Available: <https://github.com/naveensugi14/MACHINE-LEARNING>